Introduction to Python

Outline

- Introduction
- Main characteristics of the programming language
- Variables and data types
- Input and output
- Operators
- Conditional statements
- Loops
- Lists
- Functions
- A complete algorithm: the maximum value in an array
- Bitwise operators
- Python and C++ side by side

Introduction

Introduction: a second programming language

- In the chapter «Introduction to C++» we translated the pseudo-code of our algorithms into a **compiled** programming language
- In this chapter we introduce a second programming language, **Python**
- Python is **interpreted**, has a lighter syntax and is very popular both in teaching and in scientific computing

The path of the chapter

The chapter follows the same path as the chapter «Introduction to C++», so that the two languages can be compared section by section:

- the main characteristics of the language and the structure of a program
- variables, data types, input and output
- operators, conditional statements and loops
- lists, which play the role of the arrays of C++
- functions, their parameters and their return values
- a complete algorithm, the search of the maximum value in an array, written in pseudo-code and in Python
- the bitwise operators and a summary table that compares Python with C++

Executed code

Every fragment of code of the chapter has been executed with the `python3` interpreter (`Python 3.14`).

The outputs reported in the examples are the ones printed by the programs.

Indices: pseudo-code and Python

- In pseudo-code the entries of an array $a[1..n]$ are indexed from 1 to n
- In Python, as in C++, the elements of a list of size n are indexed from 0 to $n-1$
- The entry $a[i]$ of the pseudo-code corresponds to the element `a[i-1]` of the Python code

Summary

- Python is a second programming language, interpreted and with a light syntax
- The chapter follows the same path as the chapter on C++, to compare the two languages
- Pseudo-code indices go from 1 to n, Python indices from 0 to $n-1$: $a[i]$ becomes `a[i-1]`

Main characteristics of the programming language

Introduction: Main characteristics of the programming language

- In this section we see what makes Python different from C++: interpreter, dynamic typing, indentation
- We see how comments are written and how a program is structured
- These rules apply to every line of code that we will write in the rest of the chapter

Main characteristics (1/2)

- Python is an **interpreted** language: programs are executed instruction by instruction by the Python **interpreter**, without a separate compilation step
  - this makes development and debugging fast and interactive
- Python uses **dynamic typing**: variables do not need to be declared with a data type
  - the data type is determined automatically, during the execution, from the assigned value
- Python supports **functions**, enabling a structured and modular approach to programming
  - functions allow code reuse and better organization

Main characteristics (2/2)

- **Indentation is mandatory** in Python: the block structure of the code (loops, conditional statements, functions) is defined by the level of indentation, not by curly braces
  - the standard indentation is 4 spaces
- A single-line comment starts with the reserved symbol "#"
- There is no dedicated symbol for multi-line comments: a text enclosed between triple quotes """ (or ''') is a string that is not used by the program, and it is commonly used as a multi-line comment
- The same string written as the first line of a function is its **documentation string** (`docstring`)

Example: comments in Python

This is an example of a single-line comment:

```
1 # This is a comment line
```

This is an example of a multi-line comment:

```
1 """
2 This is a
3 multi-line comment
4 """
```

The structure of a program

- A Python program is a text file (with extension `.py`) that is executed by the interpreter from the first line to the last one
- Unlike C++, **no `main` function** is required: the instructions written at the top level of the file are executed in order

Example: the skeleton of a program

```
1 # The instructions of the program are written here
2 print("Hello, world!")
```

If the source code is saved in the file `hello.py`, it is executed with the command

```
python3 hello.py
```

and the output is

```
Hello, world!
```

In the following examples, unless they contain a complete program, the fragments of code are written at the top level of a file and executed in this way.

Blocks of instructions

- A **block** of instructions (the body of a conditional statement, of a loop or of a function) is introduced by the symbol " : " at the end of the preceding line
- All its instructions are indented by the same number of spaces
- The block ends at the first instruction that is indented less

Example: blocks defined by indentation

```
1 x = 7
2 if x > 0:
3     print("positive")          # Inside the block of the
      if
4     print("still inside")      # Inside the block of the
      if
5 print("outside the block")     # Executed in any case
```

Output:

```
positive
still inside
outside the block
```

In C++ the same block would be enclosed in curly braces `{}`; in Python a wrong indentation changes the meaning of the program or produces an `IndentationError`.

No semicolons, no curly braces

In Python the instructions do not end with “;”,
and the blocks are delimited by the indentation (4 spaces)
instead of curly braces.

Summary

The language:

- Interpreted, with dynamic typing and functions
- Comments with #; triple quotes """ for multi-line comments and docstrings

The structure of a program:

- A `.py` file executed from the first line to the last one, run with `python3`; no `main` function
- Blocks introduced by ":" and delimited by the indentation (4 spaces)

Variables and data types

Introduction: Variables and data types

- The source code operates on **variables**, and every value has a **data type**
- In this section we see the four fundamental data types of Python and how variables are created
- The main difference with C++: no declarations, the data type is not written in the code

The four fundamental data types (1/2)

Information is categorized into different **data types**. The four fundamental data types are:

1. The data type "`int`" represents an integer, which is a whole number written without a decimal point
   - this includes positive and negative whole numbers, as well as zero
   - unlike C++, the integers of Python have **no maximum value**: they can have as many digits as needed

2. The data type "`bool`" represents a Boolean value: `True` or `False` (note the capital first letter)

The four fundamental data types (2/2)

3. The data type "`float`" represents a floating-point number, which includes numbers written with a decimal point
   - a `float` of Python has the same (double) precision as a `double` of C++

4. The data type "`str`" represents a string of characters, enclosed in single or double quotes (e.g., `'hello'`, `"world"`)
   - there is no separate data type for a single character: a character is a string of length 1

Variables

- A variable acts as a container that stores a value. Each variable has a unique name
- Variables are assigned values using the assignment operator "=". When a new value is assigned, the previous value is overwritten
- **No declaration is needed** before use: a variable is created the moment a value is assigned to it
- The function `type()` returns the data type of a value

Variables are created by assignment

Variables are created by assignment;
there is no separate declaration step,
and the data type is not written in the code.

Example: variable assignments (1/2)

```
1  age = 25                          # Integer variable
2  is_raining = False                # Boolean variable
3  small_number = 3.14               # Floating-point variable
4  large_number = 123456.789         # Floating-point variable
5  character = 'a'                   # String variable (single
       character)
6  name = "Alice"                    # String variable
7
8  age = 26  # The previous value 25 is overwritten
9  print(age, is_raining, small_number, large_number,
       character, name)
10 print(type(age), type(is_raining), type(small_number),
       type(name))
```

Example: variable assignments (2/2)

Output:

```
26 False 3.14 123456.789 a Alice
<class 'int'> <class 'bool'> <class 'float'> <class 'str'>
```

- Compare with the example on screen output in the next section and with the corresponding example of the chapter «Introduction to C++»
- The Boolean value is displayed as `False`
- The number `123456.789` is displayed with all its digits

Example: dynamic typing and large integers

```
1 x = 5
2 print(x, type(x))
3 x = "five"  # The same name now refers to a string
4 print(x, type(x))
5
6 print(2 ** 100)  # An integer with 31 digits
```

Output:

```
5 <class 'int'>
five <class 'str'>
1267650600228229401496703205376
```

The operator ** is the exponentiation (see the arithmetic operators). In C++ the value 2^{100} does not fit in a variable of type `int`.

Summary

Data types:

- `int` (no maximum value), `bool` (`True`, `False`), `float` (double precision), `str` (a character is a string of length 1)

Variables:

- Created by assignment with "=", without declaration
- The same name can later refer to a value of another data type
- `type()` returns the data type of a value

Input and output

Introduction: Input and output

- A program communicates with the user: it displays results and reads data
- In this section we see the function `print()` for the output and the function `input()` for the input
- We also see the f-strings, which combine text and values in a compact way

The function `print()`

- The function `print()` displays output on the screen
- Multiple values can be printed in one call by separating them with commas: they are displayed separated by a space, and the line ends with a new line (the same effect as `endl` in C++)
- The optional parameters `sep` and `end` change the separator between the values and the text written at the end
  - `print(x, end=" ")` writes a space instead of a new line, so the next `print` continues on the same line

f-strings

- Strings and variable values can be combined using **f-strings** (formatted string literals), prefixed with the letter `f`
- Inside f-strings, variable names (and, more generally, expressions) are placed between curly braces `{}`
- Inside the curly braces of an f-string, the format `:.kf` displays a floating-point number with exactly k digits after the decimal point (the same effect as `fixed` and `setprecision(k)` in C++)

Example: screen output (1/2)

```
1 n = 42
2 print("The value of the variable is", n)
3 print(f"The value of the variable is {n}")  # f-string
    version
4 print(f"n = {n}, doubled = {n * 2}")        #
    Expression inside {}
5
6 print("a", "b", "c", sep="-")   # Separator "-" instead
     of a space
7 print("No new line", end=" ")   # A space instead of
    the new line
8 print("after end")
9 print(f"{10 / 3:.2f}")          # Two digits after the
    decimal point
```

Example: screen output (2/2)

Output:

```
The value of the variable is 42
The value of the variable is 42
n = 42, doubled = 84
a-b-c
No new line after end
3.33
```

The function `input()`

- The function `input()` displays a message, waits for the user to type a line of text and returns it
- It always returns a `str`
- To obtain a number, the result must be converted with `int()` or `float()`

Example: keyboard input

```
1 name = input("Enter your name: ")                    # Reads a
      string
2 n = int(input("Enter an integer: "))                 # Reads
    and converts to int
3 x = float(input("Enter a real number: "))            # Reads
    and converts to float
4 print(f"Hello {name}, n = {n}, x = {x}")
```

Example of execution (the user types `Ada`, `7` and `2.5`):

```
Enter your name: Ada
Enter an integer: 7
Enter a real number: 2.5
Hello Ada, n = 7, x = 2.5
```

Summary

Output:

- `print()` writes the values separated by a space and ends the line
- The parameters `sep` and `end` change separator and end of line
- f-strings: `f"...{expression}..."`, with `:.kf` for k decimal digits

Input:

- `input()` always returns a `str`
- Numbers are obtained with `int()` or `float()`

Operators

Introduction: Operators

- Operators are symbols that perform operations on variables and values
- As in C++, they are classified, according to their functionality, into **arithmetic**, **assignment**, **comparison** and **logical** operators
- Python has **no increment and decrement operators**
- We pay attention to the differences with C++, in particular for the division

Arithmetic operators

Arithmetic operators, used to perform basic mathematical operations:

- + (Addition) $\rightsquigarrow$ Adds two values
- - (Subtraction) $\rightsquigarrow$ Subtracts one value from another
- * (Multiplication) $\rightsquigarrow$ Multiplies two values
- / (Division) $\rightsquigarrow$ Divides one value by another (always returns a `float`)
- // (Integer division) $\rightsquigarrow$ Returns the integer quotient (floor division)
- % (Modulus) $\rightsquigarrow$ Returns the remainder of a division
- ** (Exponentiation) $\rightsquigarrow$ Raises a value to a power

Example: arithmetic operators (1/2)

```
1  a = 10
2  b = 3
3
4  print("Addition:", a + b)
5  print("Subtraction:", a - b)
6  print("Multiplication:", a * b)
7  print("Division:", a / b)              # Always a float
8  print("Integer division:", a // b)
9  print("Modulus:", a % b)
10 print("Exponentiation:", a ** b)
```

Example: arithmetic operators (2/2)

Output:

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Integer division: 3
Modulus: 1
Exponentiation: 1000
```

The last digit `5` of the division is not an error of the program: $10/3$ has infinitely many decimal digits, and the `float` closest to it is displayed.

Division: Python and C++

- The division operator / always returns a `float` in Python 3, even when dividing two integers
- To obtain an integer result, use the integer division operator //
- This is the **opposite convention** with respect to C++, where `10 / 3` is the integer division

Floor division with negative numbers

- The operator `//` computes $\lfloor a/b \rfloor$, rounding down
- For non-negative operands it coincides with the integer division of C++
- For a negative quotient it does not, because C++ truncates toward zero:
  - in Python `-7 // 2` is `-4` and `-7 % 2` is `1`
  - in C++ `-7 / 2` is `-3` and `-7 % 2` is `-1`
- In both languages the identity $a = b \cdot q + r$ holds between quotient q and remainder r

Type conversion

- A value is converted into another data type (**type conversion**) with the functions `int()`, `float()` and `str()`
- Examples:
  - `int(3.9)` is 3 (the decimal part is removed)
  - `float(7)` is `7.0`
  - `int("12")` is the integer 12

Example: integer and floating-point division

```python
a = 10
b = 3

print(a / b)                # Floating-point division
print(a // b)               # Integer division
print(f"{a / b:.2f}")       # Two decimals
print(-7 // 2, -7 % 2)      # Floor division with a negative quotient
print(int(-7 / 2))          # Truncation toward zero, as in C++
print(int(3.9), float(7), int("12") + 1)
```

Output:

```
3.3333333333333335
3
3.33
-4 1
-3
3 7.0 13
```

Assignment operators (1/2)

Assignment operators, used to assign values to variables:

- = (Assignment) $\rightsquigarrow$ Assigns a value to a variable
- += (Addition assignment) $\rightsquigarrow$ Adds a value and assigns the result
- -= (Subtraction assignment) $\rightsquigarrow$ Subtracts a value and assigns the result
- *= (Multiplication assignment) $\rightsquigarrow$ Multiplies and assigns the result

Assignment operators (2/2)

- `/=` (Division assignment) $\rightsquigarrow$ Divides and assigns the result (a `float`)
- `//=` (Integer division assignment) $\rightsquigarrow$ Floor-divides and assigns the result
- `%=` (Modulus assignment) $\rightsquigarrow$ Applies the modulus and assigns the result
- `**=` (Exponentiation assignment) $\rightsquigarrow$ Raises to a power and assigns the result

Example: assignment operators (1/3)

```
1  x = 10  # Assignment
2  print("Initial value:", x)
3
4  x += 5   # Equivalent to: x = x + 5
5  print("After += 5:", x)
6
7  x -= 3   # Equivalent to: x = x - 3
8  print("After -= 3:", x)
9
10 x *= 2   # Equivalent to: x = x * 2
11 print("After *= 2:", x)
```

Example: assignment operators (2/3)

```
12
13 x //= 4  # Equivalent to: x = x // 4
14 print("After //= 4:", x)
15
16 x %= 4   # Equivalent to: x = x % 4
17 print("After %= 4:", x)
18
19 x **= 3  # Equivalent to: x = x ** 3
20 print("After **= 3:", x)
21
22 x /= 4   # Equivalent to: x = x / 4 (the result is a
       float)
23 print("After /= 4:", x)
```

Example: assignment operators (3/3)

Output:

```
Initial value: 10
After += 5: 15
After -= 3: 12
After *= 2: 24
After //= 4: 6
After %= 4: 2
After **= 3: 8
After /= 4: 2.0
```

Increment and decrement

- Python has no operators ++ and - -
- A variable is incremented with `x += 1` and decremented with `x -= 1`
- The instruction `x++` is a syntax error
- The expression `++x` is accepted, but it does not increment `x`: it applies twice the unary plus, so its value is `x` itself

Example: increment and decrement

```
x = 5
x += 1              # Increment
print("After x += 1:", x)
x -= 1              # Decrement
print("After x -= 1:", x)
print("++x is", ++x, "and x is still", x)  # ++x is not
    an increment
```

Output:

```
After x += 1: 6
After x -= 1: 5
++x is 5 and x is still 5
```

Comparison operators

Comparison operators, used to compare values and return a Boolean result:

- == (Equal to) $\rightsquigarrow$ Checks if two values are equal
- != (Not equal to) $\rightsquigarrow$ Checks if two values are different
- < (Less than) $\rightsquigarrow$ Checks if one value is smaller than another
- > (Greater than) $\rightsquigarrow$ Checks if one value is larger than another
- <= (Less than or equal to) $\rightsquigarrow$ Checks if one value is smaller than or equal to another
- >= (Greater than or equal to) $\rightsquigarrow$ Checks if one value is larger than or equal to another

Boolean results and chained comparisons

- Comparison operators return a Boolean result: `True` if the condition is satisfied, `False` otherwise
  - displayed as `True` and `False`, not as `1` and `0` as in C++
- These operators are commonly used in **conditional statements** and **loops**
- Comparisons can be chained: `0 < x <= 10` means `0 < x` and `x <= 10`

Example: comparison operators (1/2)

```
1  a = 10
2  b = 5
3
4  print("a == b:", a == b)  # Equal to
5  print("a != b:", a != b)  # Not equal to
6  print("a < b:", a < b)    # Less than
7  print("a > b:", a > b)    # Greater than
8  print("a <= b:", a <= b)  # Less than or equal to
9  print("a >= b:", a >= b)  # Greater than or equal to
10 print("0 < b <= a:", 0 < b <= a)  # Chained comparison
```

Example: comparison operators (2/2)

Output:

```
a == b: False
a != b: True
a < b: False
a > b: True
a <= b: False
a >= b: True
0 < b <= a: True
```

Logical operators

Logical operators, used for logical operations, often in conditional statements:

- `and` (Logical AND) $\rightsquigarrow$ Returns `True` if both conditions are `True`
- `or` (Logical OR) $\rightsquigarrow$ Returns `True` if at least one condition is `True`
- `not` (Logical NOT) $\rightsquigarrow$ Reverses the Boolean value

- The logical operators of Python are **words**, while in C++ they are the symbols `&&`, `||` and `!`

Example: logical operators (1/2)

```
1  condition1 = True
2  condition2 = False
3
4  print("condition1:", condition1)
5  print("condition2:", condition2)
6
7  # Logical AND: True only if both conditions are True
8  print("condition1 and condition2:", condition1 and
       condition2)
9
10 # Logical OR: True if at least one condition is True
11 print("condition1 or condition2:", condition1 or
       condition2)
12
13 # Logical NOT: reverses the Boolean value
14 print("not condition1:", not condition1)
15 print("not condition2:", not condition2)
```

Example: logical operators (2/2)

Output:

```
condition1: True
condition2: False
condition1 and condition2: False
condition1 or condition2: True
not condition1: False
not condition2: True
```

Summary (1/2)

Arithmetic operators:

- `/` always gives a `float`; `//` is the integer division and rounds down, also for negative numbers; `**` is the exponentiation
- Type conversion with `int()`, `float()`, `str()`

Assignment, increment and decrement:

- Compound assignments `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`
- No `++` and `--`: use `x += 1` and `x -= 1`

Summary (2/2)

Comparison operators:

- `==`, `!=`, `<`, `>`, `<=`, `>=`, with results `True` and `False`
- Comparisons can be chained: `0 < x <= 10`

Logical operators:

- The words `and`, `or`, `not` instead of `&&`, `||`, `!`

Conditional statements

Introduction: Conditional statements

- The comparison and logical operators produce the conditions that decide which instructions are executed
- In this section we see the statements `if`, `elif` and `else`
- We also see the conditional expression, the Python version of the ternary operator of C++

The statements `if`, `elif` and `else`

- Conditional statements allow executing different blocks of code based on specific conditions
- The `if` statement evaluates a condition. If the condition is `True`, the indented block is executed
- The optional `elif` (else if) clause checks an additional condition when the previous ones are `False`
- The optional `else` clause is executed when all preceding conditions are `False`
- **No parentheses are needed** around the condition, and **no curly braces** are used: the line of the condition ends with " :" and the block is delimited by indentation

Example: conditional statement

```
1 number = 10
2
3 # Check if number is positive
4 if number > 0:
5     print("The number is positive.")
6 else:
7     print("The number is non-positive.")
```

Output:

```
The number is positive.
```

Example: if / elif / else (1/2)

```
1  score = 85
2
3  if score >= 90:
4      grade = "A"
5  elif score >= 80:
6      grade = "B"
7  elif score >= 70:
8      grade = "C"
9  elif score >= 60:
10     grade = "D"
11 else:
12     grade = "F"
13
14 print(f"Score: {score}, Grade: {grade}")
```

Example: `if` / `elif` / `else` (2/2)

Output:

```
Score: 85, Grade: B
```

- The conditions are evaluated from top to bottom, and only the block of the first true condition is executed
- `score >= 90` is false and `score >= 80` is true, so the grade is `"B"`
- This is the same result as the nested ternary operators of the chapter «Introduction to C++»

Conditional expression

- The **conditional expression** is the Python version of the ternary operator `?:` of C++: it provides a compact way to write simple `if-else` statements
- Syntax:

```
value_if_true if condition else value_if_false
```

- If the condition is `True`, the expression evaluates to `value_if_true`; otherwise, it evaluates to `value_if_false`

Example: conditional expression (1/3)

```python
number = 10

# Compact conditional expression
result = "positive" if number > 0 else "non-positive"
print(f"The number is {result}")

# Finding the maximum of two numbers
a, b = 15, 20  # Multiple assignment: a = 15 and b = 20
maximum = a if a > b else b
print(f"Maximum: {maximum}")
```

Example: conditional expression (2/3)

```
11
12 # Checking even or odd
13 n = 7
14 parity = "even" if n % 2 == 0 else "odd"
15 print(f"{n} is {parity}")
16
17 # Absolute value
18 x = -5
19 abs_value = x if x >= 0 else -x
20 print(f"Absolute value of {x} is {abs_value}")
```

Example: conditional expression (3/3)

Output:

```
The number is positive
Maximum: 20
7 is odd
Absolute value of -5 is 5
```

The Python library also provides the functions `max(a, b)` and `abs(x)`, which return the same values.

Summary

Conditional statements:

- `if`, then any number of `elif`, then an optional `else`
- No parentheses, no curly braces: "`:`" and indentation
- Only the block of the first true condition is executed

Conditional expression:

- `x if c else y`, the Python version of `c ? x : y`

Loops

Introduction: Loops

- Conditional statements choose *whether* a block of code is executed; loops decide *how many times* it is executed
- Loops are used to execute a block of code repeatedly
- In Python, the two loop constructs are the `while` loop and the `for` loop
- We also see the statements `break` and `continue`

The `while` loop

- The `while` loop executes a block of code as long as the condition evaluates to `True`
- If the condition is `False` from the beginning, the block is never executed

Example: `while` loop

```python
i = 1                    # Initialization

while i <= 5:            # Condition
    print(f"Number: {i}")
    i += 1               # Increment (there is no ++ operator in Python)
```

Output:

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

As in C++, the condition `i <= 5` is evaluated six times: five times it is true, and the sixth time (`i` equal to 6) it is false and the loop terminates.

The `for` loop

- The `for` loop of Python iterates over the elements of an **iterable**, such as a list, a string, or a range of integers: at each iteration the loop variable takes the next element
- The function `range(start, stop, step)` generates the sequence of integers from `start` (included) to `stop` (excluded), with increment `step`
  - if only one argument is given, the sequence starts at 0
  - if `step` is omitted, it is 1
  - with a negative `step` the sequence decreases

From pseudo-code to Python

- The pseudo-code loop

  “**for** $i := 1$ **to** n”

  corresponds to `for i in range(1, n + 1)`

- The loop over the indices of a list of size n is

  `for i in range(n)`

  with $i = 0, 1, \ldots, n - 1$

Example: `for` loop with `range`

```
for i in range(1, 6):   # i takes the values 1, 2, 3, 4, 5
    print(f"Number: {i}")
```

The output is the same as in the `while` loop of the previous example. The function `list()` shows the integers generated by a `range`:

```
print(list(range(5)))
print(list(range(1, 6)))
print(list(range(0, 10, 3)))
print(list(range(5, 0, -1)))
```

Output:

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 5]
[0, 3, 6, 9]
[5, 4, 3, 2, 1]
```

Example: `for` loop over a list

```
1 a = [3, 7, 2, 9, 5]
2
3 for element in a:
4     print(element)
```

Output:

```
3
7
2
9
5
```

The loop visits the elements directly, without using their indices.

The `break` and `continue` statements

- The `break` statement immediately terminates the loop
- The `continue` statement skips the rest of the current iteration and moves to the next one

Example: break and continue

```python
for i in range(1, 11):
    if i % 2 == 0:
        continue  # Skip the even numbers
    if i > 7:
        break     # Stop the loop at the first odd number greater than 7
    print(i, end=" ")
print()
```

Output:

```
1 3 5 7
```

- As in the C++ version, the even values of `i` are skipped by `continue`
- When `i` is equal to 9 the `break` statement terminates the loop, and the value 10 is never reached
- The final `print()` writes only the new line

Summary

The `while` loop:

- Repeats the block as long as the condition is `True`

The `for` loop:

- Iterates over the elements of an iterable: a `range`, a list, a string
- `range(start, stop, step)`: `stop` is excluded
- "**for** $i := 1$ **to** n" becomes `for i in range(1, n + 1)`

`break` **and** `continue`:

- Terminate the loop, or skip to the next iteration

Lists

Introduction: Lists

- Loops become really useful when they visit collections of values
- In Python a collection of values is stored in a **list**
- Lists play the role of the arrays of C++, and they are the data structure of the algorithms that we will write
- We also see two-dimensional lists, and a common mistake to avoid

Lists (1/2)

- A **list** is an ordered, mutable collection of elements, that can hold values of any type
  - lists are the main sequence data structure in Python and play the role of the arrays of C++
- Lists are enclosed in square brackets `[]` and elements are separated by commas
- Elements are accessed by their **index**, starting from `0`: the elements of a list `a` of size n are `a[0]`, `a[1]`, ..., `a[n-1]`
  - the function `len(a)` returns the size of the list
  - the negative index `a[-1]` denotes the last element

Lists (2/2)

- Unlike the arrays of C++, lists are **dynamic**: they can grow or shrink at runtime without manual memory management
  - the instruction `a.append(x)` adds the element `x` at the end of the list
- The expression `[0] * n` creates a list of n zeros
- More generally, a **list comprehension** `[expression for i in range(n)]` creates the list of the n values of the expression for $i = 0, 1, \ldots, n-1$
  - when the loop variable is not used, it is conventionally called `_`

Creating lists

Lists are created with `[]`
and indexed starting from `0`.

No memory allocation or deallocation is required.

Example: creating and accessing a list

```
1  n = 5
2
3  # Create a list of n zeros
4  a = [0] * n
5
6  # Assign values
7  for i in range(n):
8      a[i] = i * 2
9
10 print(a)        # The whole list
11 print(a[0])     # First element
12 print(a[4])     # Last element, index n-1
13 print(len(a))   # Number of elements
```

Output:

```
[0, 2, 4, 6, 8]
0
8
5
```

Example: list literal, modification and growth (1/2)

```
1 b = [9, 12, 3, 16]    # List of 4 integers
2
3 b[2] = 5              # Assign a new value to the third
      element
4 b.append(20)          # The list grows: now it has 5
      elements
5 print(b, len(b))
6 print(b[-1])          # The last element
7
8 squares = [i * i for i in range(6)]  # List
      comprehension
9 print(squares)
```

Example: list literal, modification and growth (2/2)

Output:

```
[9, 12, 5, 16, 20] 5
20
[0, 1, 4, 9, 16, 25]
```

Accessing an index that does not exist, for example `b[5]`, stops the program with the error `IndexError: list index out of range` (in C++ the same access is not checked).

Example: list literal and iteration over the indices

```python
a = [9, 12, 3, 16, 19, 11, 12]
n = len(a)

for i in range(n):
    print(f"a[{i}] = {a[i]}")
```

Output:

```
a[0] = 9
a[1] = 12
a[2] = 3
a[3] = 16
a[4] = 19
a[5] = 11
a[6] = 12
```

Two-dimensional lists

- Two-dimensional data can be represented as a **list of lists**
- Each row is itself a list, and elements are accessed with two indices `matrix[i][j]`, both starting from 0
  - for example, if `c = [[3, 1, 4], [1, 5, 9]]`, then `c[1][2]` is the element in the second row and third column, equal to 9
- A matrix of m rows and n columns filled with zeros is created with the list comprehension `[[0] * n for _ in range(m)]`, which builds m distinct rows

Example: two-dimensional list (matrix) (1/2)

```python
m = 3   # Number of rows
n = 2   # Number of columns

# Create a matrix of zeros
matrix = [[0] * n for _ in range(m)]

# Assign values
for i in range(m):
    for j in range(n):
        matrix[i][j] = i * j

# Print the matrix
for i in range(m):
    for j in range(n):
        print(f"matrix[{i}][{j}] = {matrix[i][j]}")
print(matrix)
```

Example: two-dimensional list (matrix) (2/2)

Output:

```
matrix[0][0] = 0
matrix[0][1] = 0
matrix[1][0] = 0
matrix[1][1] = 1
matrix[2][0] = 0
matrix[2][1] = 2
[[0, 0], [0, 1], [0, 2]]
```

Example: a common mistake with two-dimensional lists

- The shorter expression `[[0] * n] * m` is wrong: it repeats *m* times the *same* row, so a change to one row appears in all the rows

```
1 wrong = [[0] * 2] * 3                    # Three
    references to the same row
2 wrong[0][0] = 1
3 print(wrong)
4
5 right = [[0] * 2 for _ in range(3)]      # Three distinct
     rows
6 right[0][0] = 1
7 print(right)
```

Output:

```
[[1, 0], [1, 0], [1, 0]]
[[1, 0], [0, 0], [0, 0]]
```

Summary

Lists:

- Ordered, mutable, dynamic collections, written with `[]`
- Indices from `0` to `len(a)-1`; `a[-1]` is the last element
- `append` adds an element at the end; `[0] * n` and list comprehensions create lists
- An index out of range gives an `IndexError`

Two-dimensional lists:

- Lists of lists, accessed with `matrix[i][j]`
- Create them with `[[0] * n for _ in range(m)]`, never with `[[0] * n] * m`

Functions

Introduction: Functions

- As in pseudo-code, an algorithm is written as a function that receives the input and returns the output
- In this section we see how a function is defined and called in Python
- We also see what a function can do to the variables of the caller, and how it returns several values

Defining a function

- Functions allow programmers to write modular and reusable code by grouping instructions into a single unit
- A function is defined with the keyword `def`, followed by the function name and the parameter list in parentheses, ending with the symbol ":"

The parts of a function

A function in Python consists of:

1. the keyword `def` followed by the **function name**
2. a **parameter list** (optional): specifies the input values, without their data types
3. an indented **body**: the block of code executed when the function is called
4. a `return` statement (optional): returns a value to the caller
   - a function without `return` returns the special value `None`

Where the code is written (1/2)

- A function must be defined before the instruction that calls it is executed: for this reason the functions are written at the beginning of the file

- The main instructions of a Python program are typically placed at the top level of the file, or inside a block guarded by

```
if __name__ == "__main__":
```

  which ensures that the code runs only when the file is executed directly (and not when it is imported as a module by another file)

Where the code is written (2/2)

- This block plays the role of the `main` function of C++
- Python source code can also be executed online, for example at the following address: https://www.mycompiler.io/online-python-compiler

Example: functions (1/2)

```python
# Function definition
def addition(a, b):
    return a + b  # Returns the sum

# Function definition
def multiplication(a, b):
    return a * b  # Returns the product

if __name__ == "__main__":
    total = addition(5, 10)          # Function call
    print(f"Sum: {total}")

    product = multiplication(5, 10)  # Function call
    print(f"Prod: {product}")
```

Example: functions (2/2)

Output:

```
Sum: 15
Prod: 50
```

Example: function operating on a two-dimensional list (1/2)

```python
def sum_matrix(matrix, rows, columns):
    """Return the sum of all elements of a 2D list."""
    total = 0
    for i in range(rows):
        for j in range(columns):
            total += matrix[i][j]
    return total

if __name__ == "__main__":
    m, n = 3, 2

    # Build the matrix with a nested list comprehension
    matrix = [[i * j for j in range(n)] for i in range(m)]

    result = sum_matrix(matrix, m, n)
    print(f"Sum of matrix elements: {result}")
```

Example: function operating on a two-dimensional list (2/2)

Output:

```
Sum of matrix elements: 3
```

- The matrix is `[[0, 0], [0, 1], [0, 2]]`, as in the example of the two-dimensional list (matrix)
- The sum of its elements is $0 + 0 + 0 + 1 + 0 + 2 = 3$

Parameters and return values

- In C++ the parameters are passed by value or by reference
- In Python every parameter receives the *same* object as the argument of the caller
- The effect of a function on the caller depends on what the function does with it

What a function does with its parameters

- If the function **assigns a new value** to a parameter (for example `x = x + 1`), only the local name changes: the variable of the caller is not affected
  - for numbers, the behaviour is the same as the parameters passed by value in C++
- If the function **modifies the elements** of a list received as a parameter (for example `a[i] = 0`), the change is visible to the caller, as for the arrays passed to a function in C++
- A function can return **several values** at once, separated by commas
  - they are returned as a **tuple**, and the caller assigns them to several variables with a multiple assignment
  - in C++ the same result requires parameters passed by reference

Example: parameters and multiple return values (1/3)

```
1 def increment(x):
2     x = x + 1                # Changes only the local name x
3
4 def fill_with_zeros(a):
5     for i in range(len(a)):
6         a[i] = 0             # Modifies the list of the
      caller
7
8 def divide(n, m):
9     return n // m, n % m  # Returns two values (a tuple
      )
```

Example: parameters and multiple return values (2/3)

```
11 if __name__ == "__main__":
12     k = 5
13     increment(k)
14     print("After increment:", k)
15 
16     b = [4, 7, 1]
17     fill_with_zeros(b)
18     print("List:", b)
19 
20     q, r = divide(17, 5)
21     print("Quotient and remainder:", q, r)
22     print("Tuple:", divide(17, 5))
```

Example: parameters and multiple return values (3/3)

Output:

```
After increment: 5
List: [0, 0, 0]
Quotient and remainder: 3 2
Tuple: (3, 2)
```

Summary

Functions:

- `def name(parameters):`, an indented body, an optional `return` (otherwise `None`)
- Defined before they are called, at the beginning of the file
- The block `if __name__ == "__main__":` plays the role of `main`

Parameters and return values:

- Assigning to a parameter does not change the caller; modifying the elements of a list does
- Several values are returned together as a tuple

A complete algorithm: the maximum value in an array

Introduction: A complete algorithm

- We now put all the pieces together: a function, a loop, a conditional statement and a list
- We translate an algorithm of the course from pseudo-code to Python, paying attention to the indices
- We also see how to generate a test instance at random, and we execute the algorithm step by step

The problem

We consider the problem of finding
the maximum value in an array a
of $n \geq 1$ integer numbers.

- The next slide shows the pseudo-code of the algorithm `FindMaximum` of the chapter «Pseudo-code»: the entries are indexed from 1 to n

The pseudo-code

Algorithm: FindMaximum

Input : An array $a[1..n]$ of $n \geq 1$ integer numbers
Output: The maximum value in the array a

```
1 function FindMaximum(a, n)
2 |   max := a[1]
3 |   for i := 2 to n do
4 |   |   if a[i] > max then
5 |   |   |   max ← a[i]
6 |   |   end
7 |   end
8 |   return max
9 end
```

From the pseudo-code to Python

- The elements are indexed from 0 to $n-1$, so the initialization uses `a[0]` and the loop `for i in range(1, len(a))` visits the indices $1, \ldots, n-1$
- The size n is not passed as a parameter, because the function obtains it with `len(a)`
- The variable is called `max_val` and not `max`, so as not to hide the built-in function `max()`
- The function assumes $n \geq 1$, as the pseudo-code does: for an empty list the access to `a[0]` stops the program with an `IndexError`

The Python implementation (1/2)

```python
def array_max_value(a):
    """Return the maximum value in the list a (len(a) >= 1)."""
    max_val = a[0]  # Initialize with the first element

    for i in range(1, len(a)):
        if a[i] > max_val:
            max_val = a[i]  # Update the maximum

    return max_val

if __name__ == "__main__":
    a = [9, 12, 3, 16, 19, 11, 12]

    max_val = array_max_value(a)
    print(f"Maximum value: {max_val}")
```

The Python implementation (2/2)

Output:

```
Maximum value: 19
```

A random test instance

- The test instance can also be generated at random with the module `random` of the Python library
- `random.randint(1, 100)` returns a random integer between 1 and 100 (both included)
- The instruction `random.seed(2026)` fixes the starting point of the random generator, so that every execution produces the same list
- Without it, every execution produces a different list

Example: a random instance (1/2)

```python
import random

def array_max_value(a):
    max_val = a[0]
    for i in range(1, len(a)):
        if a[i] > max_val:
            max_val = a[i]
    return max_val

if __name__ == "__main__":
    random.seed(2026)  # Same random list at every execution
    n = 10             # Size of the list
    a = [random.randint(1, 100) for _ in range(n)]

    print(a)
    print(f"Maximum value: {array_max_value(a)}")
```

Example: a random instance (2/2)

Output:

```
[16, 41, 65, 66, 83, 14, 29, 77, 80, 72]
Maximum value: 83
```

Example: step-by-step execution (1/3)

- We execute the function `array_max_value` on two instances
- In each table, the column i is the index of the Python loop and the column $i + 1$ is the corresponding index of the pseudo-code
- The first row is the initialization `max_val = a[0]`

Example: step-by-step execution (2/3)

Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$, $n = 7$

i (Python)	$i+1$ (pseudo-code)	`a[i]`	`a[i] > max_val?`	`max_val`
–	–	–	–	9
1	2	12	Yes	12
2	3	3	No	12
3	4	16	Yes	16
4	5	19	Yes	19
5	6	11	No	19
6	7	12	No	19

Return: `max_val` $= 19$

Example: step-by-step execution (3/3)

Instance 2: $a = [3, 7, 2, 9, 1]$, $n = 5$

i (Python)	$i+1$ (pseudo-code)	`a[i]`	`a[i] > max_val?`	`max_val`
–	–	–	–	3
1	2	7	Yes	7
2	3	2	No	7
3	4	9	Yes	9
4	5	1	No	9

Return: `max_val` $= 9$

Summary

- The pseudo-code `FindMaximum` becomes the function `array_max_value`
- $a[1]$ becomes `a[0]`, and "**for** $i := 2$ **to** n" becomes `for i in range(1, len(a))`
- The size is obtained with `len(a)`; the name `max_val` does not hide the built-in `max()`
- Random instances with `random.randint`; `random.seed` makes them reproducible
- The step-by-step tables match each Python index i with the pseudo-code index $i + 1$

Bitwise operators

Introduction: Bitwise operators

- The bitwise operators of the chapter «Introduction to C++» exist in Python with the same symbols and the same meaning
- In this section we list them and we see the only difference: the bitwise NOT
- The difference comes from the integers of Python, which have no fixed number of bits

The bitwise operators

- `&` (AND)
- `|` (OR)
- `^` (XOR)
- `~` (NOT)
- `<<` (left shift)
- `>>` (right shift)

The function `bin()` returns the binary representation of an integer, preceded by the prefix `0b`.

The bitwise NOT

- Since the integers of Python have no fixed number of bits, the bitwise NOT does not depend on a data type
- `~x` is always equal to $-x-1$
- To obtain the 8-bit result of C++, the result is combined with the mask `0xFF` (the number 255, whose binary representation is 11111111)

Example: bitwise operators (1/2)

```
a = 12  # Binary: 1100
b = 10  # Binary: 1010

print(a & b, a | b, a ^ b)           # AND, OR, XOR
print(bin(a), bin(b), bin(a & b))    # Binary
    representations
print(5 << 2, 20 >> 2, 20 >> 3)      # Shifts
print(~a, ~a & 0xFF)                 # NOT, and its 8-
    bit version

n = 15
print("odd" if n & 1 else "even")    # Least significant
     bit
```

Example: bitwise operators (2/2)

Output:

```
8 14 6
0b1100 0b1010 0b1000
20 5 2
-13 243
odd
```

All the values coincide with those of the corresponding examples of the chapter «Introduction to C++».

Summary

- `&`, `|`, `^`, `~`, `<<`, `>>`: the same symbols and the same meaning as in C++
- `bin()` gives the binary representation, with the prefix `0b`
- `~x` $= -x - 1$; the 8-bit result of C++ is `~x & 0xFF`

Python and C++ side by side

Introduction: Python and C++ side by side

- We have met the same constructs in two languages
- The following table summarizes the main differences between the two languages met in this chapter
- It is a quick reference to use when we move from one language to the other

Python and C++ (1/3)

	C++	Python
Execution	compiled (g++), then executed	interpreted (`python3`)
Data types	declared: `int x = 5;`	not declared: `x = 5`
End of instruction	`;`	end of the line
Blocks	curly braces `{}`	`:` and indentation
Comments	`//` and `/* */`	`#` (and `""" """`)
Boolean values	`true`, `false` (printed `1`, `0`)	`True`, `False`

Python and C++ (2/3)

	C++	Python
`10 / 3`	3 (integer division)	`3.3333333333333335`
Integer division	`/` between two `int`	`//`
Power	function `pow`	operator `**`
Increment	`x++`, `++x`	`x += 1`
Logical operators	`&&`, `||`, `!`	`and`, `or`, `not`
Ternary operator	`c ? x : y`	`x if c else y`

Python and C++ (3/3)

	C++	Python
`for` loop	`for (int i = 0; i < n; i++)`	`for i in range(n):`
Collections	arrays, fixed or dynamic size	lists, dynamic
Size of a collection	stored in a separate variable	`len(a)`
Several results	parameters by reference	return of a tuple
Entry point	function `main`	top level of the file

Summary

- C++ is compiled and declares the data types; Python is interpreted and does not
- Python replaces “;” and curly braces with the end of the line, “:” and the indentation
- Division, increment, logical and ternary operators are written differently
- Lists are dynamic and know their size; several results are returned as a tuple

Summary (1/2)

Key points of the chapter:

- A Python program is executed by the interpreter (`python3`) from the first line to the last one; the instructions do not end with "`;`" and the blocks are delimited by "`:`" and by the indentation
- Variables are created by assignment, without declaring their data type; the integers have no maximum value
- The operator `/` always gives a `float`; the integer division is `//` and it rounds down, also for negative numbers

Summary (2/2)

- Conditional statements (`if`, `elif`, `else`, and the conditional expression) choose which instructions are executed; loops (`while`, and `for` over a `range` or a list) repeat them
- The lists of Python are indexed from 0 to $n-1$, with $n =$ `len(a)`, while the arrays of the pseudo-code are indexed from 1 to n; lists grow with `append` and need no memory management
- A function modifies the list of the caller when it changes its elements, but not a number it receives; several results are returned together as a tuple